

Running Stata in a Jupyter notebook on Google Colab or repo2docker



Figure 1: Open In Colab

This notebook installs Stata into the notebook runtime, asks for **your** Stata license (as a base64-encoded string), and then runs Stata code through the `%stata` cell magic provided by StataCorp's PyStata.

Before you start, create the base64-encoded version of your `stata.lic` on your own computer:

```
base64 -w0 /usr/local/stata/stata.lic      # Linux
base64 -i /Applications/Stata/stata.lic    # macOS
```

```
[Convert]::ToBase64String([IO.File]::ReadAllBytes("C:\Program
Files\Stata19\stata.lic"))    # Windows
```

Copy the resulting single line; you will paste it into a hidden prompt below. It is only written to the runtime's Stata directory, never to the notebook.

1. Settings

Stata is copied out of one of the AEA Data Editor's Stata Docker images (which contain no license). Set the version using the image-name spelling (e.g. `19_5` for Stata 19.5) and pick a tag, or use `latest` to look up the most recently updated tag on Docker Hub. Pick the edition that matches your license: an MP license can also run SE, so `se` works for both SE and MP licenses.

```
import os

VERSION = "19_5"          # image-name spelling, e.g. "19_5" for Stata 19.5
TAG = "2026-08-12"        # a specific tag, or "latest"
EDITION = "se"            # "be", "se" or "mp"
IMAGE = f"dataeditors/stata{VERSION}-{EDITION}:{TAG}"
STATA_ROOT = "/" if os.geteuid() == 0 else os.path.expanduser("~")
STATA_DIR = os.path.join(STATA_ROOT, "usr/local/stata")
```

2. Install Stata

This uses the helper script from this repository (downloading it only if needed) and copies Stata to `/usr/local/stata` on Colab, or `~/usr/local/stata` for non-root users such as `repo2docker` (about 700 MB, usually well under a minute on Colab).

```
if not os.path.exists("setup_stata.py"):
    !wget -q https://raw.githubusercontent.com/larsvilhuber/jupyter-stata-colab/
main/setup_stata.py

import setup_stata

setup_stata.install_stata(image=IMAGE, root=STATA_ROOT)
```

```
Fetching Stata from docker.io/dataeditors/stata19_5-se:2026-08-12 ...
layer 1/9 sha256:b71466b94f26
layer 2/9 sha256:50bbe9355d9a
layer 3/9 sha256:36b113ab1d27
layer 4/9 sha256:21f5f60104a3
layer 5/9 sha256:c2114ca3083a
layer 6/9 sha256:2f76430387b3
layer 7/9 sha256:bae8c00e3552
layer 8/9 sha256:f5762a497a42
layer 9/9 sha256:5df1c422ab3d
Stata installed in /usr/local/stata
```

```
'/usr/local/stata'
```

3. Enter your license

Paste the base64-encoded content of your `stata.lic` when prompted (the input is hidden). If the environment variable `STATA_LIC_BASE64` is set, it is used instead of prompting.

```
from getpass import getpass

license_b64 = os.environ.get("STATA_LIC_BASE64") or getpass("Base64-encoded
stata.lic: ")
setup_stata.write_license(license_b64, stata_dir=STATA_DIR)
del license_b64
```

```
License written to /usr/local/stata/stata.lic
```

4. Start Stata

This first checks that Stata accepts the license, then initializes PyStata, which registers the `%stata` and `%%stata` magics.

```
setup_stata.configure_pystata(stata_dir=STATA_DIR, edition=EDITION)
```

Stata license OK.



StataNow 19.5
SE—Standard Edition

Statistics and Data Science

Copyright 1985-2025 StataCorp LLC
StataCorp
4905 Lakeway Drive
College Station, Texas 77845 USA
800-782-8272 <https://www.stata.com>
979-696-4600 service@stata.com

Stata license: 2-user network, expiring 27 Apr 2027

Serial number: 501909230155

Licensed to: Lars Vilhuber / AEA Data Editor
Cornell University

Notes:

1. Unicode is supported; see help `unicode_advice`.
2. Maximum number of variables is set to 5,000 but can be increased; see help `set_maxvar`.

```
<module 'pystata.config' from '/usr/local/stata/utilities/pystata/config.py'>
```

5. Use Stata

Any cell starting with `%stata` is run by Stata.

```
%stata
sysuse auto, clear
summarize price mpg weight
```

```
. sysuse auto, clear
(1978 automobile data)
```

```
. summarize price mpg weight
```

Variable	Obs	Mean	Std. dev.	Min	Max
price	74	6165.257	2949.496	3291	15906
mpg	74	21.2973	5.785503	12	41
weight	74	3019.459	777.1936	1760	4840

```
.
```

```
%%stata
```

```
regress price mpg weight foreign
```

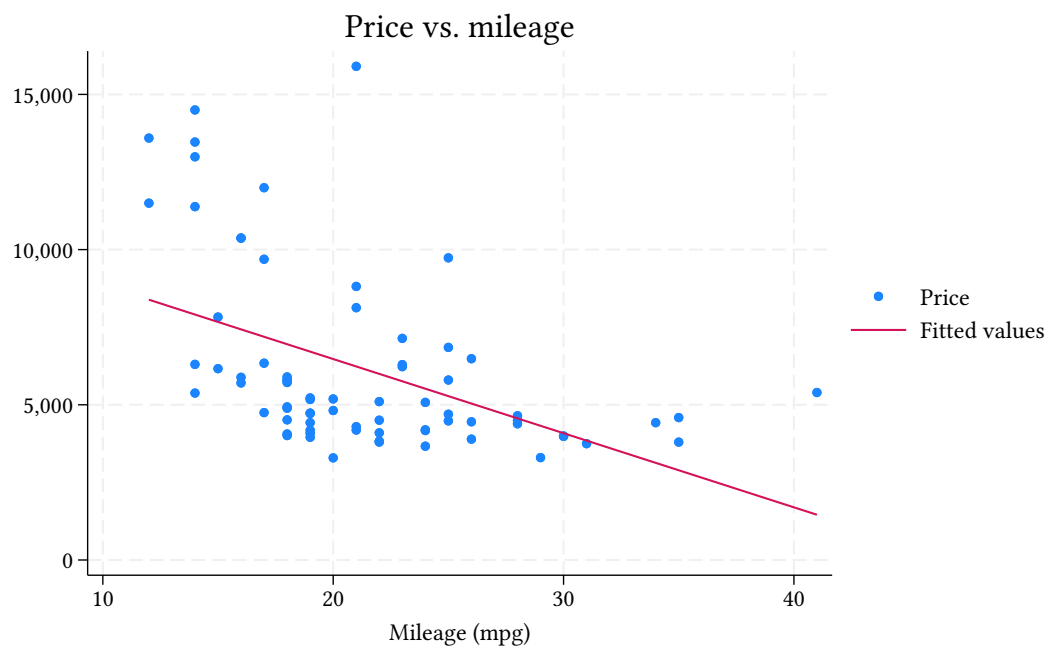
Source	SS	df	MS	Number of obs	=	74
Model	317252881	3	105750960	F(3, 70)	=	23.29
Residual	317812515	70	4540178.78	Prob > F	=	0.0000
Total	635065396	73	8699525.97	R-squared	=	0.4996
				Adj R-squared	=	0.4781
				Root MSE	=	2130.8

price	Coefficient	Std. err.	t	P> t	[95% conf. interval]
mpg	21.8536	74.22114	0.29	0.769	-126.1758 169.883
weight	3.464706	.630749	5.49	0.000	2.206717 4.722695
foreign	3673.06	683.9783	5.37	0.000	2308.909 5037.212
_cons	-5853.696	3376.987	-1.73	0.087	-12588.88 881.4934

Graphs are displayed inline:

```
%%stata
```

```
twoway (scatter price mpg) (lfit price mpg), title("Price vs. mileage")
```



Data can be moved between Stata and Python (pandas):

```
from pystata import stata

df = stata.pdataframe_from_data()
df.groupby("foreign")[["price", "mpg"]].mean()
```

	price	mpg
foreign		
0	6072.423077	19.826923
1	6384.681818	24.772727

```
%%stata -d df -force
describe, short
```

```
Contains data
Observations:      74
Variables:         12
Sorted by:
Note: Dataset has changed since last saved.
```

6. Your turn: a printable table and a figure

The cells below are the solutions to part 4 of the Reproducible Documents tutorial: a clean regression table and a figure, a number from Stata in a sentence, and a figure made with Python. Run the cells above first.

A clean, printable regression table (`etable`), and a figure that is shown below the cell and saved to disk.

```
%%stata
* Estimate three models and store them
sysuse auto, clear
quietly regress price mpg
estimates store m1
quietly regress price mpg weight
estimates store m2
quietly regress price mpg weight i.foreign
estimates store m3

* A clean, printable regression table
etable, estimates(m1 m2 m3) column(index) varlabel ///
    showstars showstarsnote mstat(N) mstat(r2)      ///
    title("Table 1: Car prices and fuel efficiency")

* A figure: shown below the cell, and saved to disk
tway (scatter price mpg) (lfit price mpg), by(foreign, note("")) ///
    ytitle("Price (USD)") xtitle("Mileage (mpg)")
graph export price_mpg.png, replace width(1600)
```

```
. * Estimate three models and store them
. sysuse auto, clear
(1978 automobile data)

. quietly regress price mpg

. estimates store m1

. quietly regress price mpg weight

. estimates store m2

. quietly regress price mpg weight i.foreign
```

```
. estimates store m3
```

```
.
. * A clean, printable regression table
. etable, estimates(m1 m2 m3) column(index) varlabel ///
> showstars showstarsnote mstat(N) mstat(r2) ///
> title("Table 1: Car prices and fuel efficiency")
```

Table 1: Car prices and fuel efficiency

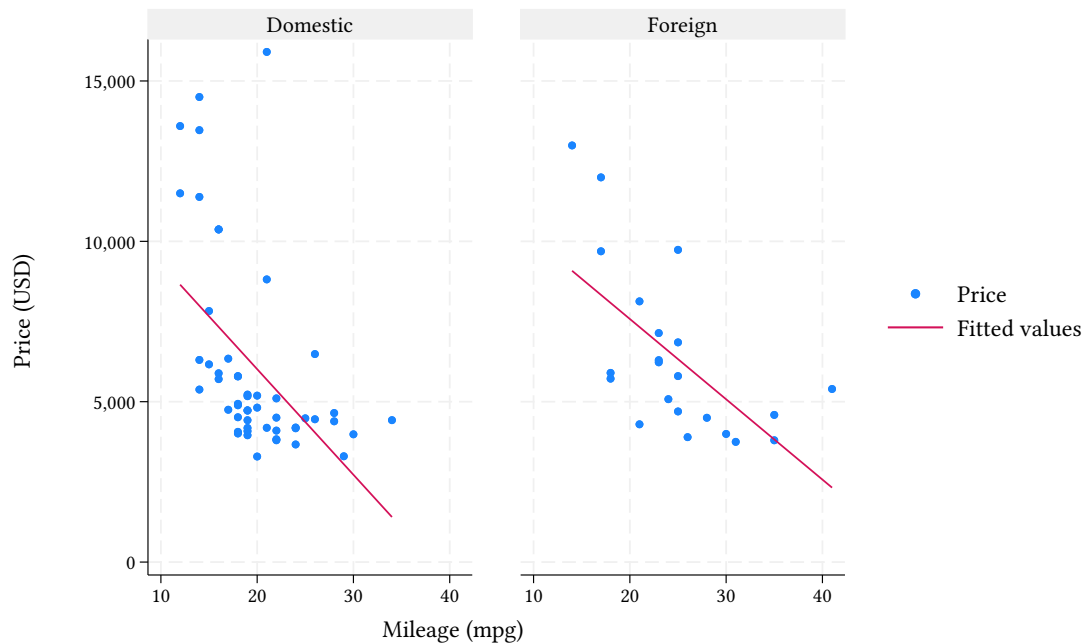
	1	2	3
Mileage (mpg)	-238.894 ** (53.077)	-49.512 (86.156)	21.854 (74.221)
Weight (lbs.)		1.747 ** (0.641)	3.465 ** (0.631)
Car origin			
Foreign			3673.060 ** (683.978)
Intercept	11253.061 ** (1170.813)	1946.069 (3597.050)	-5853.696 (3376.987)
Number of observations	74	74	74
R-squared	0.22	0.29	0.50

** p<.01, * p<.05

```
.
. * A figure: shown below the cell, and saved to disk
. twoway (scatter price mpg) (lfit price mpg), by(foreign, note("")) ///
> ytitle("Price (USD)") xtitle("Mileage (mpg)")
```

```
. graph export price_mpg.png, replace width(1600)
(file price_mpg.png not found)
file price_mpg.png written in PNG format
```

```
.
```



A number from Stata, in a sentence: Python asks Stata for the coefficient, and writes the sentence (as Markdown) below the cell. No `%%stata` here: this is Python.

```
# A number from Stata, in a sentence (Python, no %%stata)
from pystata import stata
from sfi import Scalar
from IPython.display import Markdown

stata.run("sysuse auto, clear", quietly=True)
stata.run("regress price mpg weight", quietly=True)
stata.run("scalar b_mpg = _b[mpg]")
b_mpg = Scalar.getValue("b_mpg")

Markdown(f"Holding weight constant, one more mile per gallon "
         f"changes the price by **{b_mpg:.1f}** dollars.")
```

Holding weight constant, one more mile per gallon changes the price by **-49.5** dollars.

A figure that is hard to make in Stata, made with Python (matplotlib) from the Stata data: the scatter plot, with a histogram of each variable along its axis.

```
# A figure that is hard to make in Stata (Python, matplotlib):
# the scatter plot, with a histogram of each variable along its axis
import matplotlib.pyplot as plt
```

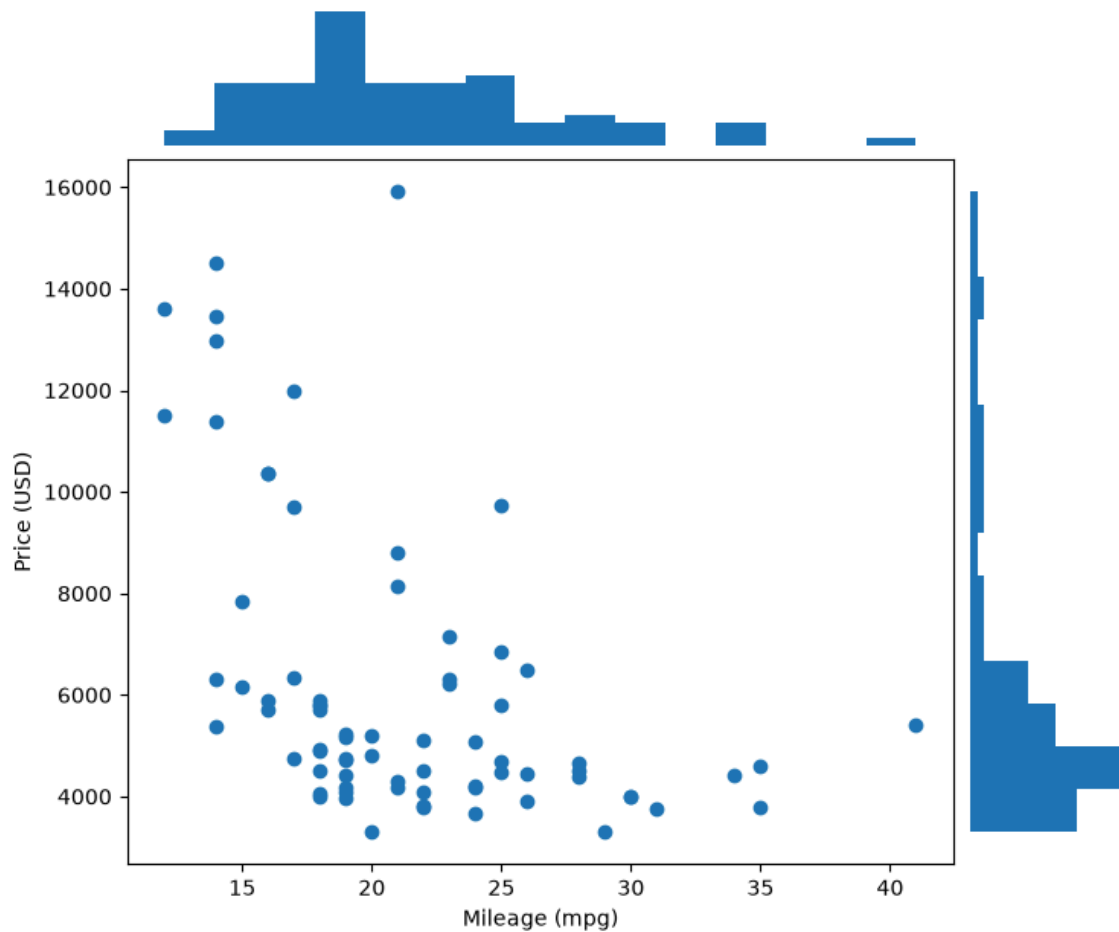
```

from pystata import stata

stata.run("sysuse auto, clear", quietly=True)
auto = stata.pdataframe_from_data()      # Stata data -> pandas

fig, ax = plt.subplots(figsize=(7, 6))
ax.scatter(auto["mpg"], auto["price"])
ax.set_xlabel("Mileage (mpg)")
ax.set_ylabel("Price (USD)")
top = ax.inset_axes([0, 1.02, 1, 0.2], sharex=ax)      # above
right = ax.inset_axes([1.02, 0, 0.2, 1], sharey=ax)    # on the right
top.hist(auto["mpg"], bins=15)
right.hist(auto["price"], bins=15, orientation="horizontal")
top.axis("off")
right.axis("off")
fig.savefig("price_mpg_hist.png", dpi=200, bbox_inches="tight")

```



7. Print this notebook

- **PDF:** *File ► Print ► Save as PDF*.
- **Word** (or PDF) on **Binder**: save the notebook, then *File ► New ► Terminal*, and type `pip install quarto-cli`, then `quarto render stata_colab_solutions.ipynb --to docx` (or `--to typst` for PDF). Add `-M echo:false` to hide the code.
- **Word** on **Colab**: *File ► Download ► Download .ipynb*, then run the same `quarto render` command wherever Quarto is (posit.cloud, or your laptop).